

Group14_Source_Code6

May 24, 2026

1 Hybrid Convolutional Transformer Network with Sequence Embedding and Bootstrap Resampling

```
[1]: # This Python 3 environment comes with many helpful analytics libraries
      ↳ installed
      # It is defined by the kaggle/python Docker image: https://github.com/kaggle/
      ↳ docker-python
      # For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list
↳ all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that
↳ gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved
↳ outside of the current session

# Use the kagglehub client library to attach Kaggle resources like
↳ competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/
↳ README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/rohitbme23b064/chess-project/chess_tensor.npy
/kaggle/input/datasets/rohitbme23b064/chess-project/chess_scalar.parquet
```

```
[ ]: # =====
# CHESS HYBRID CNN + TRANSFORMER MODEL
# =====
# Architecture:
#
# Tensor Input (12x8x8)
#     ↓
# CNN Feature Extraction
#     ↓
# Reshape to Sequence
#     ↓
# Transformer Encoder
#     ↓
# Global Pooling
#     ↓
# Concatenate Scalar Features
#     ↓
# Dense Layers
#     ↓
# Softmax Output
#
# Includes:
# - Oversampling
# - Memory optimization
# - Batch feature handling
# - Model saving
# - Graphs
# - Confusion matrix
# - Classification report
# - Random prediction samples
# =====

# =====
# IMPORTS
# =====

import os
import gc
import joblib

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.utils import resample
```

```

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import (
    LabelEncoder,
    StandardScaler
)

from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)

import tensorflow as tf

from tensorflow.keras import (
    layers,
    models,
    regularizers,
    callbacks
)

# =====
# ENABLE GPU MEMORY GROWTH
# =====

gpus = tf.config.experimental.list_physical_devices('GPU')

for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)

print("\nGPUs Available:\n")
print(gpus)

# =====
# DATASET PATH
# =====

base_path = "/kaggle/input/datasets/sachin211104/sachin-chess-project/"

# =====
# LOAD DATA
# =====

base_path = "/kaggle/input/datasets/rohitbme23b064/chess-project/"

```

```

# =====
# LOAD DATA
# =====

print("\nLoading dataframe...\n")

df = pd.read_parquet(
    base_path + "chess_scalar.parquet"
)

print("Scalar shape:", df.shape)

print("\nLoading tensor...\n")

X_tensor = np.load(
    base_path + "chess_tensor.npy",
    mmap_mode='r'
)

print("Tensor shape:", X_tensor.shape)
#-----

# =====
# ORIGINAL CLASS COUNTS
# =====

print("\n===== ORIGINAL CLASS COUNTS =====\n")

print(
    df["label"].value_counts()
)

# =====
# CREATE INDEX
# =====

df = df.reset_index(drop=True)

df["idx"] = np.arange(len(df))

# =====
# OVERSAMPLING
# =====

print("\nPerforming oversampling...\n")

```

```

max_count = df["label"].value_counts().max()

balanced_indices = []

for label_name in df["label"].unique():

    class_indices = df[
        df["label"] == label_name
    ]["idx"].values

    oversampled_indices = resample(

        class_indices,

        replace=True,

        n_samples=max_count,

        random_state=42
    )

    balanced_indices.extend(
        oversampled_indices
    )

balanced_indices = np.array(
    balanced_indices
)

np.random.shuffle(
    balanced_indices
)

# =====
# BUILD BALANCED DATASET
# =====

df_balanced = df.iloc[
    balanced_indices
].reset_index(drop=True)

# lazy tensor loading
X_tensor_balanced = X_tensor[
    balanced_indices
]

# =====

```

```

# BALANCED COUNTS
# =====

print("\n===== BALANCED CLASS COUNTS =====\n")

print(
    df_balanced["label"].value_counts()
)

# =====
# LABEL ENCODING
# =====

le = LabelEncoder()

y_encoded = le.fit_transform(
    df_balanced["label"]
)

y = tf.keras.utils.to_categorical(
    y_encoded
)

print("\nClasses:\n")
print(le.classes_)

# =====
# REMOVE LEAKAGE FEATURES
# =====

leak_cols = [
    "label",
    "delta",
    "eval",
    "piece_moved",
    "is_capture",
    "gives_check",
    "idx"
]

X_scalar = df_balanced.drop(
    columns=leak_cols
).values

print("\nScalar shape:")
print(X_scalar.shape)

```

```

# =====
# TRAIN TEST SPLIT
# =====

X_tensor_train, X_tensor_test, \
X_scalar_train, X_scalar_test, \
y_train, y_test = train_test_split(

    X_tensor_balanced,
    X_scalar,
    y,

    test_size=0.2,

    random_state=42,

    stratify=np.argmax(y, axis=1)
)

# =====
# SCALE SCALAR FEATURES
# =====

scaler = StandardScaler()

X_scalar_train = scaler.fit_transform(
    X_scalar_train
)

X_scalar_test = scaler.transform(
    X_scalar_test
)

# =====
# MEMORY OPTIMIZATION
# =====

X_tensor_train = X_tensor_train.astype(
    np.float16
)

X_tensor_test = X_tensor_test.astype(
    np.float16
)

X_scalar_train = X_scalar_train.astype(

```

```

        np.float32
    )

X_scalar_test = X_scalar_test.astype(
    np.float32
)

# =====
# LABELS
# =====

y_train_labels = np.argmax(
    y_train,
    axis=1
)

y_test_labels = np.argmax(
    y_test,
    axis=1
)

print("\nTrain tensor:")
print(X_tensor_train.shape)

print("\nTest tensor:")
print(X_tensor_test.shape)

# =====
# POSITIONAL EMBEDDING LAYER
# =====

class PositionalEmbedding(
    layers.Layer
):

    def __init__(self, sequence_length, d_model):

        super().__init__()

        self.pos_embedding = layers.Embedding(
            input_dim=sequence_length,
            output_dim=d_model
        )

        self.sequence_length = sequence_length
        self.d_model = d_model

```

```

def call(self, inputs):

    positions = tf.range(
        start=0,
        limit=self.sequence_length,
        delta=1
    )

    embedded_positions = self.pos_embedding(
        positions
    )

    return inputs + embedded_positions

# =====
# TRANSFORMER ENCODER BLOCK
# =====

def transformer_encoder(
    inputs,
    head_size,
    num_heads,
    ff_dim,
    dropout=0.1
):

    x = layers.MultiHeadAttention(
        key_dim=head_size,
        num_heads=num_heads,
        dropout=dropout
    )(inputs, inputs)

    x = layers.Dropout(dropout)(x)

    x = layers.LayerNormalization(
        epsilon=1e-6
    )(x)

    res = x + inputs

    x = layers.Dense(
        ff_dim,
        activation='swish'
    )(res)

    x = layers.Dropout(dropout)(x)

```

```

x = layers.Dense(
    inputs.shape[-1]
)(x)

x = layers.LayerNormalization(
    epsilon=1e-6
)(x)

return x + res

# =====
# BUILD MODEL
# =====

def build_model():

    # =====
    # TENSOR INPUT
    # =====

    tensor_input = layers.Input(
        shape=(12,8,8)
    )

    x = layers.Permute(
        (2,3,1)
    )(tensor_input)

    # =====
    # CNN BLOCK
    # =====

    x = layers.Conv2D(
        32,
        (3,3),
        padding='same',
        activation='swish'
    )(x)

    x = layers.BatchNormalization()(x)

    x = layers.Conv2D(
        64,
        (3,3),
        padding='same',
        activation='swish'
    )(x)

```

```

x = layers.BatchNormalization()(x)

# =====
# RESHAPE TO SEQUENCE
# =====

x = layers.Reshape(
    (64,64)
)(x)

# =====
# POSITIONAL EMBEDDING
# =====

x = PositionalEmbedding(
    sequence_length=64,
    d_model=64
)(x)

# =====
# TRANSFORMER BLOCKS
# =====

x = transformer_encoder(
    x,
    head_size=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1
)

x = transformer_encoder(
    x,
    head_size=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1
)

# =====
# GLOBAL POOLING
# =====

x = layers.GlobalAveragePooling1D()(x)

# =====

```

```

# SCALAR INPUT
# =====

scalar_input = layers.Input(
    shape=(X_scalar_train.shape[1],)
)

y = layers.Dense(
    64,
    activation='swish'
)(scalar_input)

y = layers.BatchNormalization()(y)

y = layers.Dropout(0.2)(y)

# =====
# CONCATENATION
# =====

combined = layers.concatenate(
    [x, y]
)

# =====
# FINAL DENSE BLOCK
# =====

z = layers.Dense(
    128,
    activation='swish',
    kernel_regularizer=regularizers.l2(0.001)
)(combined)

z = layers.Dropout(0.3)(z)

z = layers.Dense(
    64,
    activation='swish'
)(z)

z = layers.Dropout(0.3)(z)

output = layers.Dense(
    len(le.classes_),
    activation='softmax'
)(z)

```

```

# =====
# MODEL
# =====

model = models.Model(

    inputs=[
        tensor_input,
        scalar_input
    ],

    outputs=output
)

optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0005,
    clipnorm=1.0
)

model.compile(

    optimizer=optimizer,

    loss='categorical_crossentropy',

    metrics=['accuracy']
)

return model

# =====
# BUILD MODEL
# =====

model = build_model()

# =====
# MODEL SUMMARY
# =====

model.summary()

# =====
# CALLBACKS
# =====

```

```

early_stop = callbacks.EarlyStopping(

    monitor='val_loss',

    patience=10,

    restore_best_weights=True
)

checkpoint = callbacks.ModelCheckpoint(

    filepath="/kaggle/working/best_transformer_model.keras",

    monitor='val_accuracy',

    save_best_only=True,

    verbose=1
)

reduce_lr = callbacks.ReduceLROnPlateau(

    monitor='val_loss',

    factor=0.5,

    patience=3,

    verbose=1
)

# =====
# TRAIN MODEL
# =====

print("\nStarting training...\n")

history = model.fit(

    [X_tensor_train, X_scalar_train],

    y_train,

    validation_split=0.2,

    epochs=100,

```

```

        batch_size=32,

        callbacks=[
            early_stop,
            checkpoint,
            reduce_lr
        ],

        verbose=1
    )

# =====
# SAVE FINAL MODEL
# =====

model.save(
    "/kaggle/working/final_transformer_model.keras"
)

joblib.dump(
    scaler,
    "/kaggle/working/scaler.pkl"
)

joblib.dump(
    le,
    "/kaggle/working/label_encoder.pkl"
)

# =====
# TRAINING CURVES
# =====

fig, (ax1, ax2) = plt.subplots(
    1,
    2,
    figsize=(14,5)
)

# LOSS

ax1.plot(
    history.history['loss'],
    label='Train Loss'
)

ax1.plot(

```

```

        history.history['val_loss'],
        label='Validation Loss'
    )

    ax1.set_title(
        'Loss Curve'
    )

    ax1.legend()

    ax1.grid(True)

    # ACCURACY

    ax2.plot(
        history.history['accuracy'],
        label='Train Accuracy'
    )

    ax2.plot(
        history.history['val_accuracy'],
        label='Validation Accuracy'
    )

    ax2.set_title(
        'Accuracy Curve'
    )

    ax2.legend()

    ax2.grid(True)

    plt.tight_layout()

    plt.show()

    # =====
    # EVALUATION
    # =====

    print("\nEvaluating model...\n")

    test_loss, test_acc = model.evaluate(

        [X_tensor_test, X_scalar_test],

        y_test,

```

```

        verbose=0
    )

    print(f"\nTest Accuracy: {test_acc*100:.2f}%")

    # =====
    # PREDICTIONS
    # =====

    y_pred_probs = model.predict(

        [X_tensor_test, X_scalar_test],

        batch_size=256,

        verbose=1
    )

    y_pred = np.argmax(
        y_pred_probs,
        axis=1
    )

    # =====
    # CONFUSION MATRIX
    # =====

    class_names = [
        str(cls)
        for cls in le.classes_
    ]

    cm = confusion_matrix(
        y_test_labels,
        y_pred
    )

    plt.figure(figsize=(8,6))

    sns.heatmap(

        cm,

        annot=True,

        fmt='d',

```

```

        cmap='Blues',

        xticklabels=class_names,

        yticklabels=class_names
    )

plt.title(
    "Confusion Matrix"
)

plt.xlabel(
    "Predicted"
)

plt.ylabel(
    "Actual"
)

plt.tight_layout()

plt.show()

# =====
# CLASSIFICATION REPORT
# =====

print("\nClassification Report:\n")

print(
    classification_report(
        y_test_labels,
        y_pred,
        target_names=class_names
    )
)

# =====
# RANDOM 100 TEST SAMPLES
# =====

random_indices = np.random.choice(

```

```

    len(y_test_labels),

    100,

    replace=False
)

actual_labels = le.inverse_transform(
    y_test_labels[random_indices]
)

predicted_labels = le.inverse_transform(
    y_pred[random_indices]
)

scalar_original = scaler.inverse_transform(
    X_scalar_test[random_indices]
)

feature_names = df_balanced.drop(
    columns=leak_cols
).columns.tolist()

sample_df = pd.DataFrame(

    scalar_original,

    columns=feature_names
)

sample_df["Actual"] = actual_labels

sample_df["Predicted"] = predicted_labels

sample_df["Correct"] = (

    sample_df["Actual"] ==
    sample_df["Predicted"]
)

# =====
# SAMPLE ACCURACY
# =====

sample_accuracy = sample_df[
    "Correct"

```

```

].mean()

print(
    f"\nRandom 100 Sample Accuracy: {sample_accuracy*100:.2f}%"
)

print("\nRandom Samples:\n")

print(sample_df.head(20))

# =====
# SAVE REPORT FILES
# =====

sample_df.to_csv(

    "/kaggle/working/random_test_predictions.csv",

    index=False
)

# =====
# SAVE TRAINING HISTORY
# =====

history_df = pd.DataFrame(
    history.history
)

history_df.to_csv(

    "/kaggle/working/training_history.csv",

    index=False
)

# =====
# FINAL SUMMARY
# =====

print("\n=====")

print("TRAINING COMPLETED SUCCESSFULLY")

print("Saved files:")

print("/kaggle/working/final_transformer_model.keras")

```

```

print("/kaggle/working/best_transformer_model.keras")
print("/kaggle/working/scaler.pkl")
print("/kaggle/working/label_encoder.pkl")
print("/kaggle/working/random_test_predictions.csv")
print("/kaggle/working/training_history.csv")

print("=====")

```

```

2026-05-23 08:20:46.763572: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1779524446.789680      745 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1779524446.798753      745 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1779524446.818882      745 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1779524446.818900      745 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1779524446.818902      745 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1779524446.818904      745 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.

```

GPUs Available:

```

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'),
PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU')]

```

Loading dataframe...

Scalar shape: (337981, 21)

Loading tensor...

Tensor shape: (337981, 12, 8, 8)

===== ORIGINAL CLASS COUNTS =====

```
label
good          221151
excellent     57825
inaccuracy    28194
mistake       22817
blunder       7994
Name: count, dtype: int64
```

Performing oversampling...

===== BALANCED CLASS COUNTS =====

```
label
good          221151
blunder       221151
excellent     221151
inaccuracy    221151
mistake       221151
Name: count, dtype: int64
```

Classes:

```
['blunder' 'excellent' 'good' 'inaccuracy' 'mistake']
```

```
Scalar shape:
(1105755, 15)
```

```
Train tensor:
(884604, 12, 8, 8)
```

```
Test tensor:
(221151, 12, 8, 8)
```

```
I0000 00:00:1779524463.986282      745 gpu_device.cc:2019] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 13757 MB memory:  -> device:
0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1779524463.991495      745 gpu_device.cc:2019] Created device
/job:localhost/replica:0/task:0/device:GPU:1 with 13757 MB memory:  -> device:
1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer	(None, 12, 8, 8)	0	-

(InputLayer)			
permute (Permute)	(None, 8, 8, 12)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 8, 8, 32)	3,488	permute[0][0]
batch_normalization (BatchNormalizatio...	(None, 8, 8, 32)	128	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 8, 8, 64)	18,496	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 8, 8, 64)	256	conv2d_1[0][0]
reshape (Reshape)	(None, 64, 64)	0	batch_normalizat...
positional_embeddi... (PositionalEmbeddi...	(None, 64, 64)	4,096	reshape[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 64, 64)	66,368	positional_embed... positional_embed...
dropout_1 (Dropout)	(None, 64, 64)	0	multi_head_atten...
layer_normalization (LayerNormalizatio...	(None, 64, 64)	128	dropout_1[0][0]
add (Add)	(None, 64, 64)	0	layer_normalizat... positional_embed...
dense (Dense)	(None, 64, 128)	8,320	add[0][0]
dropout_2 (Dropout)	(None, 64, 128)	0	dense[0][0]
dense_1 (Dense)	(None, 64, 64)	8,256	dropout_2[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 64, 64)	128	dense_1[0][0]
add_1 (Add)	(None, 64, 64)	0	layer_normalizat... add[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 64, 64)	66,368	add_1[0][0], add_1[0][0]
dropout_4 (Dropout)	(None, 64, 64)	0	multi_head_atten...
layer_normalizatio...	(None, 64, 64)	128	dropout_4[0][0]

(LayerNormalizatio...			
add_2 (Add)	(None, 64, 64)	0	layer_normalizat... add_1[0][0]
dense_2 (Dense)	(None, 64, 128)	8,320	add_2[0][0]
dropout_5 (Dropout)	(None, 64, 128)	0	dense_2[0][0]
dense_3 (Dense)	(None, 64, 64)	8,256	dropout_5[0][0]
input_layer_1 (InputLayer)	(None, 15)	0	-
layer_normalizatio... (LayerNormalizatio...	(None, 64, 64)	128	dense_3[0][0]
dense_4 (Dense)	(None, 64)	1,024	input_layer_1[0]...
add_3 (Add)	(None, 64, 64)	0	layer_normalizat... add_2[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 64)	256	dense_4[0][0]
global_average_poo... (GlobalAveragePool...	(None, 64)	0	add_3[0][0]
dropout_6 (Dropout)	(None, 64)	0	batch_normalizat...
concatenate (Concatenate)	(None, 128)	0	global_average_p... dropout_6[0][0]
dense_5 (Dense)	(None, 128)	16,512	concatenate[0][0]
dropout_7 (Dropout)	(None, 128)	0	dense_5[0][0]
dense_6 (Dense)	(None, 64)	8,256	dropout_7[0][0]
dropout_8 (Dropout)	(None, 64)	0	dense_6[0][0]
dense_7 (Dense)	(None, 5)	325	dropout_8[0][0]

Total params: 219,237 (856.39 KB)

Trainable params: 218,917 (855.14 KB)

Non-trainable params: 320 (1.25 KB)

Starting training...

Epoch 1/100

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1779524481.040384 814 service.cc:152] XLA service 0x7dddb000e680 initialized for platform CUDA (this does not guarantee that XLA will be used).

Devices:

I0000 00:00:1779524481.040429 814 service.cc:160] StreamExecutor device (0): Tesla T4, Compute Capability 7.5

I0000 00:00:1779524481.040435 814 service.cc:160] StreamExecutor device (1): Tesla T4, Compute Capability 7.5

I0000 00:00:1779524483.190966 814 cuda_dnn.cc:529] Loaded cuDNN version 91002

21/22116 3:04 8ms/step -
accuracy: 0.2531 - loss: 1.8964

I0000 00:00:1779524492.920388 814 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

22116/22116 0s 6ms/step -
accuracy: 0.4738 - loss: 1.2160

Epoch 1: val_accuracy improved from -inf to 0.52986, saving model to /kaggle/working/best_transformer_model.keras

22116/22116 179s 7ms/step
- accuracy: 0.4739 - loss: 1.2160 - val_accuracy: 0.5299 - val_loss: 1.0781 -
learning_rate: 5.0000e-04

Epoch 2/100

22111/22116 0s 6ms/step -
accuracy: 0.5579 - loss: 1.0329

Epoch 2: val_accuracy improved from 0.52986 to 0.57050, saving model to /kaggle/working/best_transformer_model.keras

22116/22116 144s 7ms/step
- accuracy: 0.5579 - loss: 1.0329 - val_accuracy: 0.5705 - val_loss: 0.9918 -
learning_rate: 5.0000e-04

Epoch 3/100

22114/22116 0s 6ms/step -
accuracy: 0.5813 - loss: 0.9748

Epoch 3: val_accuracy did not improve from 0.57050

22116/22116 144s 6ms/step
- accuracy: 0.5813 - loss: 0.9748 - val_accuracy: 0.5704 - val_loss: 0.9896 -
learning_rate: 5.0000e-04

Epoch 4/100

```

22115/22116          0s 6ms/step -
accuracy: 0.5957 - loss: 0.9432
Epoch 4: val_accuracy improved from 0.57050 to 0.59086, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 6ms/step
- accuracy: 0.5957 - loss: 0.9432 - val_accuracy: 0.5909 - val_loss: 0.9539 -
learning_rate: 5.0000e-04
Epoch 5/100
22116/22116          0s 6ms/step -
accuracy: 0.6038 - loss: 0.9192
Epoch 5: val_accuracy improved from 0.59086 to 0.59092, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6038 - loss: 0.9192 - val_accuracy: 0.5909 - val_loss: 0.9415 -
learning_rate: 5.0000e-04
Epoch 6/100
22113/22116          0s 6ms/step -
accuracy: 0.6108 - loss: 0.9019
Epoch 6: val_accuracy did not improve from 0.59092
22116/22116          145s 7ms/step
- accuracy: 0.6108 - loss: 0.9019 - val_accuracy: 0.5893 - val_loss: 0.9534 -
learning_rate: 5.0000e-04
Epoch 7/100
22111/22116          0s 6ms/step -
accuracy: 0.6156 - loss: 0.8877
Epoch 7: val_accuracy improved from 0.59092 to 0.61435, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 6ms/step
- accuracy: 0.6156 - loss: 0.8877 - val_accuracy: 0.6143 - val_loss: 0.8815 -
learning_rate: 5.0000e-04
Epoch 8/100
22113/22116          0s 6ms/step -
accuracy: 0.6181 - loss: 0.8777
Epoch 8: val_accuracy did not improve from 0.61435
22116/22116          144s 6ms/step
- accuracy: 0.6181 - loss: 0.8777 - val_accuracy: 0.6058 - val_loss: 0.8979 -
learning_rate: 5.0000e-04
Epoch 9/100
22108/22116          0s 6ms/step -
accuracy: 0.6226 - loss: 0.8703
Epoch 9: val_accuracy did not improve from 0.61435
22116/22116          143s 6ms/step
- accuracy: 0.6226 - loss: 0.8703 - val_accuracy: 0.6079 - val_loss: 0.9075 -
learning_rate: 5.0000e-04
Epoch 10/100
22111/22116          0s 6ms/step -
accuracy: 0.6256 - loss: 0.8639
Epoch 10: val_accuracy improved from 0.61435 to 0.62198, saving model to

```

```

/kaggle/working/best_transformer_model.keras
22116/22116          144s 6ms/step
- accuracy: 0.6256 - loss: 0.8639 - val_accuracy: 0.6220 - val_loss: 0.8643 -
learning_rate: 5.0000e-04
Epoch 11/100
22112/22116          0s 6ms/step -
accuracy: 0.6306 - loss: 0.8549
Epoch 11: val_accuracy improved from 0.62198 to 0.62600, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          143s 6ms/step
- accuracy: 0.6306 - loss: 0.8549 - val_accuracy: 0.6260 - val_loss: 0.8597 -
learning_rate: 5.0000e-04
Epoch 12/100
22109/22116          0s 6ms/step -
accuracy: 0.6307 - loss: 0.8527
Epoch 12: val_accuracy improved from 0.62600 to 0.62746, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          143s 6ms/step
- accuracy: 0.6307 - loss: 0.8527 - val_accuracy: 0.6275 - val_loss: 0.8579 -
learning_rate: 5.0000e-04
Epoch 13/100
22112/22116          0s 6ms/step -
accuracy: 0.6346 - loss: 0.8474
Epoch 13: val_accuracy improved from 0.62746 to 0.63159, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6346 - loss: 0.8474 - val_accuracy: 0.6316 - val_loss: 0.8540 -
learning_rate: 5.0000e-04
Epoch 14/100
22108/22116          0s 6ms/step -
accuracy: 0.6370 - loss: 0.8429
Epoch 14: val_accuracy improved from 0.63159 to 0.63256, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          145s 7ms/step
- accuracy: 0.6370 - loss: 0.8429 - val_accuracy: 0.6326 - val_loss: 0.8525 -
learning_rate: 5.0000e-04
Epoch 15/100
22109/22116          0s 6ms/step -
accuracy: 0.6393 - loss: 0.8400
Epoch 15: val_accuracy improved from 0.63256 to 0.63794, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          145s 7ms/step
- accuracy: 0.6393 - loss: 0.8400 - val_accuracy: 0.6379 - val_loss: 0.8347 -
learning_rate: 5.0000e-04
Epoch 16/100
22115/22116          0s 6ms/step -
accuracy: 0.6429 - loss: 0.8344
Epoch 16: val_accuracy improved from 0.63794 to 0.64043, saving model to

```

```

/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6429 - loss: 0.8344 - val_accuracy: 0.6404 - val_loss: 0.8374 -
learning_rate: 5.0000e-04
Epoch 17/100
22116/22116          0s 6ms/step -
accuracy: 0.6455 - loss: 0.8315
Epoch 17: val_accuracy improved from 0.64043 to 0.64685, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6455 - loss: 0.8315 - val_accuracy: 0.6468 - val_loss: 0.8233 -
learning_rate: 5.0000e-04
Epoch 18/100
22109/22116          0s 6ms/step -
accuracy: 0.6473 - loss: 0.8293
Epoch 18: val_accuracy did not improve from 0.64685
22116/22116          144s 7ms/step
- accuracy: 0.6473 - loss: 0.8293 - val_accuracy: 0.6290 - val_loss: 0.8836 -
learning_rate: 5.0000e-04
Epoch 19/100
22108/22116          0s 6ms/step -
accuracy: 0.6505 - loss: 0.8253
Epoch 19: val_accuracy improved from 0.64685 to 0.65431, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6505 - loss: 0.8253 - val_accuracy: 0.6543 - val_loss: 0.8136 -
learning_rate: 5.0000e-04
Epoch 20/100
22113/22116          0s 6ms/step -
accuracy: 0.6545 - loss: 0.8205
Epoch 20: val_accuracy did not improve from 0.65431
22116/22116          144s 7ms/step
- accuracy: 0.6545 - loss: 0.8205 - val_accuracy: 0.6518 - val_loss: 0.8168 -
learning_rate: 5.0000e-04
Epoch 21/100
22111/22116          0s 6ms/step -
accuracy: 0.6564 - loss: 0.8184
Epoch 21: val_accuracy improved from 0.65431 to 0.66110, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          145s 7ms/step
- accuracy: 0.6564 - loss: 0.8184 - val_accuracy: 0.6611 - val_loss: 0.8053 -
learning_rate: 5.0000e-04
Epoch 22/100
22109/22116          0s 6ms/step -
accuracy: 0.6581 - loss: 0.8169
Epoch 22: val_accuracy did not improve from 0.66110
22116/22116          145s 7ms/step
- accuracy: 0.6581 - loss: 0.8169 - val_accuracy: 0.6538 - val_loss: 0.8116 -

```

```

learning_rate: 5.0000e-04
Epoch 23/100
22113/22116          0s 6ms/step -
accuracy: 0.6595 - loss: 0.8136
Epoch 23: val_accuracy improved from 0.66110 to 0.66598, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          145s 7ms/step
- accuracy: 0.6595 - loss: 0.8136 - val_accuracy: 0.6660 - val_loss: 0.7957 -
learning_rate: 5.0000e-04
Epoch 24/100
22112/22116          0s 6ms/step -
accuracy: 0.6629 - loss: 0.8083
Epoch 24: val_accuracy improved from 0.66598 to 0.66739, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6629 - loss: 0.8083 - val_accuracy: 0.6674 - val_loss: 0.8016 -
learning_rate: 5.0000e-04
Epoch 25/100
22115/22116          0s 6ms/step -
accuracy: 0.6657 - loss: 0.8058
Epoch 25: val_accuracy improved from 0.66739 to 0.67027, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6657 - loss: 0.8058 - val_accuracy: 0.6703 - val_loss: 0.7905 -
learning_rate: 5.0000e-04
Epoch 26/100
22114/22116          0s 6ms/step -
accuracy: 0.6679 - loss: 0.8007
Epoch 26: val_accuracy did not improve from 0.67027
22116/22116          144s 7ms/step
- accuracy: 0.6679 - loss: 0.8007 - val_accuracy: 0.6686 - val_loss: 0.7963 -
learning_rate: 5.0000e-04
Epoch 27/100
22115/22116          0s 6ms/step -
accuracy: 0.6707 - loss: 0.7981
Epoch 27: val_accuracy improved from 0.67027 to 0.67090, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          145s 7ms/step
- accuracy: 0.6707 - loss: 0.7981 - val_accuracy: 0.6709 - val_loss: 0.7873 -
learning_rate: 5.0000e-04
Epoch 28/100
22114/22116          0s 6ms/step -
accuracy: 0.6695 - loss: 0.8006
Epoch 28: val_accuracy improved from 0.67090 to 0.67758, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          144s 7ms/step
- accuracy: 0.6695 - loss: 0.8006 - val_accuracy: 0.6776 - val_loss: 0.7766 -
learning_rate: 5.0000e-04

```

Epoch 29/100
 22110/22116 0s 6ms/step -
 accuracy: 0.6728 - loss: 0.7939
 Epoch 29: val_accuracy improved from 0.67758 to 0.67961, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 145s 7ms/step
 - accuracy: 0.6728 - loss: 0.7939 - val_accuracy: 0.6796 - val_loss: 0.7700 -
 learning_rate: 5.0000e-04
 Epoch 30/100
 22113/22116 0s 6ms/step -
 accuracy: 0.6746 - loss: 0.7914
 Epoch 30: val_accuracy did not improve from 0.67961
 22116/22116 144s 7ms/step
 - accuracy: 0.6746 - loss: 0.7914 - val_accuracy: 0.6777 - val_loss: 0.7835 -
 learning_rate: 5.0000e-04
 Epoch 31/100
 22115/22116 0s 6ms/step -
 accuracy: 0.6753 - loss: 0.7904
 Epoch 31: val_accuracy improved from 0.67961 to 0.68227, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 144s 7ms/step
 - accuracy: 0.6753 - loss: 0.7904 - val_accuracy: 0.6823 - val_loss: 0.7715 -
 learning_rate: 5.0000e-04
 Epoch 32/100
 22111/22116 0s 6ms/step -
 accuracy: 0.6758 - loss: 0.7895
 Epoch 32: val_accuracy did not improve from 0.68227

 Epoch 32: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
 22116/22116 145s 7ms/step
 - accuracy: 0.6758 - loss: 0.7895 - val_accuracy: 0.6760 - val_loss: 0.7821 -
 learning_rate: 5.0000e-04
 Epoch 33/100
 22110/22116 0s 6ms/step -
 accuracy: 0.6917 - loss: 0.7533
 Epoch 33: val_accuracy improved from 0.68227 to 0.70224, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 144s 6ms/step
 - accuracy: 0.6917 - loss: 0.7533 - val_accuracy: 0.7022 - val_loss: 0.7199 -
 learning_rate: 2.5000e-04
 Epoch 34/100
 22112/22116 0s 6ms/step -
 accuracy: 0.6994 - loss: 0.7313
 Epoch 34: val_accuracy improved from 0.70224 to 0.70346, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 144s 7ms/step
 - accuracy: 0.6994 - loss: 0.7313 - val_accuracy: 0.7035 - val_loss: 0.7197 -
 learning_rate: 2.5000e-04

Epoch 35/100
 22115/22116 0s 6ms/step -
 accuracy: 0.7041 - loss: 0.7237
 Epoch 35: val_accuracy improved from 0.70346 to 0.70838, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 146s 7ms/step
 - accuracy: 0.7041 - loss: 0.7237 - val_accuracy: 0.7084 - val_loss: 0.7076 -
 learning_rate: 2.5000e-04
 Epoch 36/100
 22108/22116 0s 6ms/step -
 accuracy: 0.7075 - loss: 0.7168
 Epoch 36: val_accuracy did not improve from 0.70838
 22116/22116 150s 7ms/step
 - accuracy: 0.7075 - loss: 0.7168 - val_accuracy: 0.7054 - val_loss: 0.7183 -
 learning_rate: 2.5000e-04
 Epoch 37/100
 22114/22116 0s 6ms/step -
 accuracy: 0.7087 - loss: 0.7137
 Epoch 37: val_accuracy improved from 0.70838 to 0.71648, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 151s 7ms/step
 - accuracy: 0.7087 - loss: 0.7137 - val_accuracy: 0.7165 - val_loss: 0.6941 -
 learning_rate: 2.5000e-04
 Epoch 38/100
 22110/22116 0s 6ms/step -
 accuracy: 0.7089 - loss: 0.7147
 Epoch 38: val_accuracy did not improve from 0.71648
 22116/22116 151s 7ms/step
 - accuracy: 0.7089 - loss: 0.7147 - val_accuracy: 0.7161 - val_loss: 0.6948 -
 learning_rate: 2.5000e-04
 Epoch 39/100
 22112/22116 0s 6ms/step -
 accuracy: 0.7110 - loss: 0.7096
 Epoch 39: val_accuracy improved from 0.71648 to 0.71754, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 152s 7ms/step
 - accuracy: 0.7110 - loss: 0.7096 - val_accuracy: 0.7175 - val_loss: 0.6892 -
 learning_rate: 2.5000e-04
 Epoch 40/100
 22116/22116 0s 6ms/step -
 accuracy: 0.7119 - loss: 0.7077
 Epoch 40: val_accuracy improved from 0.71754 to 0.72175, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 152s 7ms/step
 - accuracy: 0.7119 - loss: 0.7077 - val_accuracy: 0.7217 - val_loss: 0.6805 -
 learning_rate: 2.5000e-04
 Epoch 41/100
 22108/22116 0s 6ms/step -

```

accuracy: 0.7140 - loss: 0.7055
Epoch 41: val_accuracy did not improve from 0.72175
22116/22116      152s 7ms/step
- accuracy: 0.7140 - loss: 0.7055 - val_accuracy: 0.7122 - val_loss: 0.6971 -
learning_rate: 2.5000e-04
Epoch 42/100
22110/22116      0s 6ms/step -
accuracy: 0.7150 - loss: 0.7030
Epoch 42: val_accuracy improved from 0.72175 to 0.72475, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      152s 7ms/step
- accuracy: 0.7150 - loss: 0.7030 - val_accuracy: 0.7248 - val_loss: 0.6789 -
learning_rate: 2.5000e-04
Epoch 43/100
22116/22116      0s 6ms/step -
accuracy: 0.7158 - loss: 0.7016
Epoch 43: val_accuracy did not improve from 0.72475
22116/22116      151s 7ms/step
- accuracy: 0.7158 - loss: 0.7016 - val_accuracy: 0.7231 - val_loss: 0.6832 -
learning_rate: 2.5000e-04
Epoch 44/100
22114/22116      0s 6ms/step -
accuracy: 0.7165 - loss: 0.6995
Epoch 44: val_accuracy improved from 0.72475 to 0.72570, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      152s 7ms/step
- accuracy: 0.7165 - loss: 0.6995 - val_accuracy: 0.7257 - val_loss: 0.6790 -
learning_rate: 2.5000e-04
Epoch 45/100
22114/22116      0s 6ms/step -
accuracy: 0.7166 - loss: 0.6999
Epoch 45: val_accuracy improved from 0.72570 to 0.72755, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      152s 7ms/step
- accuracy: 0.7166 - loss: 0.6999 - val_accuracy: 0.7275 - val_loss: 0.6696 -
learning_rate: 2.5000e-04
Epoch 46/100
22109/22116      0s 6ms/step -
accuracy: 0.7191 - loss: 0.6976
Epoch 46: val_accuracy did not improve from 0.72755
22116/22116      152s 7ms/step
- accuracy: 0.7191 - loss: 0.6976 - val_accuracy: 0.7262 - val_loss: 0.6791 -
learning_rate: 2.5000e-04
Epoch 47/100
22114/22116      0s 6ms/step -
accuracy: 0.7189 - loss: 0.6958
Epoch 47: val_accuracy improved from 0.72755 to 0.73075, saving model to
/kaggle/working/best_transformer_model.keras

```

```

22116/22116          153s 7ms/step
- accuracy: 0.7189 - loss: 0.6958 - val_accuracy: 0.7307 - val_loss: 0.6688 -
learning_rate: 2.5000e-04
Epoch 48/100
22108/22116          0s 6ms/step -
accuracy: 0.7193 - loss: 0.6949
Epoch 48: val_accuracy did not improve from 0.73075
22116/22116          153s 7ms/step
- accuracy: 0.7193 - loss: 0.6949 - val_accuracy: 0.7297 - val_loss: 0.6684 -
learning_rate: 2.5000e-04
Epoch 49/100
22113/22116          0s 6ms/step -
accuracy: 0.7219 - loss: 0.6896
Epoch 49: val_accuracy did not improve from 0.73075
22116/22116          151s 7ms/step
- accuracy: 0.7219 - loss: 0.6896 - val_accuracy: 0.7307 - val_loss: 0.6659 -
learning_rate: 2.5000e-04
Epoch 50/100
22112/22116          0s 6ms/step -
accuracy: 0.7225 - loss: 0.6891
Epoch 50: val_accuracy improved from 0.73075 to 0.73094, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          151s 7ms/step
- accuracy: 0.7225 - loss: 0.6891 - val_accuracy: 0.7309 - val_loss: 0.6696 -
learning_rate: 2.5000e-04
Epoch 51/100
22108/22116          0s 6ms/step -
accuracy: 0.7216 - loss: 0.6911
Epoch 51: val_accuracy did not improve from 0.73094
22116/22116          150s 7ms/step
- accuracy: 0.7216 - loss: 0.6911 - val_accuracy: 0.7244 - val_loss: 0.6864 -
learning_rate: 2.5000e-04
Epoch 52/100
22110/22116          0s 6ms/step -
accuracy: 0.7220 - loss: 0.6883
Epoch 52: val_accuracy did not improve from 0.73094
22116/22116          151s 7ms/step
- accuracy: 0.7220 - loss: 0.6883 - val_accuracy: 0.7308 - val_loss: 0.6658 -
learning_rate: 2.5000e-04
Epoch 53/100
22113/22116          0s 6ms/step -
accuracy: 0.7233 - loss: 0.6873
Epoch 53: val_accuracy improved from 0.73094 to 0.73217, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          150s 7ms/step
- accuracy: 0.7233 - loss: 0.6873 - val_accuracy: 0.7322 - val_loss: 0.6653 -
learning_rate: 2.5000e-04
Epoch 54/100

```

```

22111/22116          0s 6ms/step -
accuracy: 0.7235 - loss: 0.6842
Epoch 54: val_accuracy did not improve from 0.73217
22116/22116          151s 7ms/step
- accuracy: 0.7235 - loss: 0.6842 - val_accuracy: 0.7313 - val_loss: 0.6635 -
learning_rate: 2.5000e-04
Epoch 55/100
22110/22116          0s 6ms/step -
accuracy: 0.7251 - loss: 0.6831
Epoch 55: val_accuracy did not improve from 0.73217
22116/22116          154s 7ms/step
- accuracy: 0.7251 - loss: 0.6831 - val_accuracy: 0.7314 - val_loss: 0.6663 -
learning_rate: 2.5000e-04
Epoch 56/100
22112/22116          0s 6ms/step -
accuracy: 0.7245 - loss: 0.6842
Epoch 56: val_accuracy improved from 0.73217 to 0.73504, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          153s 7ms/step
- accuracy: 0.7245 - loss: 0.6842 - val_accuracy: 0.7350 - val_loss: 0.6557 -
learning_rate: 2.5000e-04
Epoch 57/100
22110/22116          0s 6ms/step -
accuracy: 0.7248 - loss: 0.6842
Epoch 57: val_accuracy improved from 0.73504 to 0.73712, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          153s 7ms/step
- accuracy: 0.7248 - loss: 0.6842 - val_accuracy: 0.7371 - val_loss: 0.6561 -
learning_rate: 2.5000e-04
Epoch 58/100
22116/22116          0s 6ms/step -
accuracy: 0.7264 - loss: 0.6816
Epoch 58: val_accuracy improved from 0.73712 to 0.73802, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          152s 7ms/step
- accuracy: 0.7264 - loss: 0.6816 - val_accuracy: 0.7380 - val_loss: 0.6510 -
learning_rate: 2.5000e-04
Epoch 59/100
22115/22116          0s 6ms/step -
accuracy: 0.7264 - loss: 0.6805
Epoch 59: val_accuracy did not improve from 0.73802
22116/22116          152s 7ms/step
- accuracy: 0.7264 - loss: 0.6805 - val_accuracy: 0.7340 - val_loss: 0.6579 -
learning_rate: 2.5000e-04
Epoch 60/100
22111/22116          0s 6ms/step -
accuracy: 0.7261 - loss: 0.6799
Epoch 60: val_accuracy did not improve from 0.73802

```

```

22116/22116          151s 7ms/step
- accuracy: 0.7261 - loss: 0.6799 - val_accuracy: 0.7361 - val_loss: 0.6550 -
learning_rate: 2.5000e-04
Epoch 61/100
22108/22116          0s 6ms/step -
accuracy: 0.7261 - loss: 0.6778
Epoch 61: val_accuracy improved from 0.73802 to 0.74077, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          151s 7ms/step
- accuracy: 0.7261 - loss: 0.6778 - val_accuracy: 0.7408 - val_loss: 0.6432 -
learning_rate: 2.5000e-04
Epoch 62/100
22109/22116          0s 6ms/step -
accuracy: 0.7275 - loss: 0.6788
Epoch 62: val_accuracy did not improve from 0.74077
22116/22116          151s 7ms/step
- accuracy: 0.7275 - loss: 0.6788 - val_accuracy: 0.7346 - val_loss: 0.6572 -
learning_rate: 2.5000e-04
Epoch 63/100
22111/22116          0s 6ms/step -
accuracy: 0.7271 - loss: 0.6781
Epoch 63: val_accuracy did not improve from 0.74077
22116/22116          152s 7ms/step
- accuracy: 0.7271 - loss: 0.6781 - val_accuracy: 0.7401 - val_loss: 0.6486 -
learning_rate: 2.5000e-04
Epoch 64/100
22116/22116          0s 6ms/step -
accuracy: 0.7275 - loss: 0.6777
Epoch 64: val_accuracy did not improve from 0.74077

Epoch 64: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
22116/22116          151s 7ms/step
- accuracy: 0.7275 - loss: 0.6777 - val_accuracy: 0.7383 - val_loss: 0.6518 -
learning_rate: 2.5000e-04
Epoch 65/100
22108/22116          0s 6ms/step -
accuracy: 0.7356 - loss: 0.6571
Epoch 65: val_accuracy improved from 0.74077 to 0.75503, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          152s 7ms/step
- accuracy: 0.7356 - loss: 0.6571 - val_accuracy: 0.7550 - val_loss: 0.6103 -
learning_rate: 1.2500e-04
Epoch 66/100
22114/22116          0s 6ms/step -
accuracy: 0.7413 - loss: 0.6421
Epoch 66: val_accuracy did not improve from 0.75503
22116/22116          156s 7ms/step
- accuracy: 0.7413 - loss: 0.6421 - val_accuracy: 0.7537 - val_loss: 0.6139 -

```

```

learning_rate: 1.2500e-04
Epoch 67/100
22116/22116          0s 6ms/step -
accuracy: 0.7429 - loss: 0.6379
Epoch 67: val_accuracy did not improve from 0.75503
22116/22116          153s 7ms/step
- accuracy: 0.7429 - loss: 0.6379 - val_accuracy: 0.7550 - val_loss: 0.6072 -
learning_rate: 1.2500e-04
Epoch 68/100
22115/22116          0s 6ms/step -
accuracy: 0.7433 - loss: 0.6353
Epoch 68: val_accuracy improved from 0.75503 to 0.75906, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          152s 7ms/step
- accuracy: 0.7433 - loss: 0.6353 - val_accuracy: 0.7591 - val_loss: 0.6002 -
learning_rate: 1.2500e-04
Epoch 69/100
22114/22116          0s 6ms/step -
accuracy: 0.7439 - loss: 0.6356
Epoch 69: val_accuracy did not improve from 0.75906
22116/22116          152s 7ms/step
- accuracy: 0.7439 - loss: 0.6356 - val_accuracy: 0.7578 - val_loss: 0.6067 -
learning_rate: 1.2500e-04
Epoch 70/100
22114/22116          0s 6ms/step -
accuracy: 0.7462 - loss: 0.6306
Epoch 70: val_accuracy improved from 0.75906 to 0.76147, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          153s 7ms/step
- accuracy: 0.7462 - loss: 0.6306 - val_accuracy: 0.7615 - val_loss: 0.5964 -
learning_rate: 1.2500e-04
Epoch 71/100
22110/22116          0s 6ms/step -
accuracy: 0.7469 - loss: 0.6309
Epoch 71: val_accuracy improved from 0.76147 to 0.76306, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          153s 7ms/step
- accuracy: 0.7469 - loss: 0.6309 - val_accuracy: 0.7631 - val_loss: 0.5910 -
learning_rate: 1.2500e-04
Epoch 72/100
22109/22116          0s 6ms/step -
accuracy: 0.7465 - loss: 0.6313
Epoch 72: val_accuracy did not improve from 0.76306
22116/22116          153s 7ms/step
- accuracy: 0.7465 - loss: 0.6313 - val_accuracy: 0.7616 - val_loss: 0.5946 -
learning_rate: 1.2500e-04
Epoch 73/100
22113/22116          0s 6ms/step -

```

```

accuracy: 0.7472 - loss: 0.6293
Epoch 73: val_accuracy did not improve from 0.76306
22116/22116      152s 7ms/step
- accuracy: 0.7472 - loss: 0.6293 - val_accuracy: 0.7592 - val_loss: 0.6024 -
learning_rate: 1.2500e-04
Epoch 74/100
22115/22116      0s 6ms/step -
accuracy: 0.7477 - loss: 0.6303
Epoch 74: val_accuracy did not improve from 0.76306

Epoch 74: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
22116/22116      152s 7ms/step
- accuracy: 0.7477 - loss: 0.6303 - val_accuracy: 0.7626 - val_loss: 0.5931 -
learning_rate: 1.2500e-04
Epoch 75/100
22114/22116      0s 6ms/step -
accuracy: 0.7499 - loss: 0.6220
Epoch 75: val_accuracy improved from 0.76306 to 0.76566, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      152s 7ms/step
- accuracy: 0.7499 - loss: 0.6220 - val_accuracy: 0.7657 - val_loss: 0.5832 -
learning_rate: 6.2500e-05
Epoch 76/100
22116/22116      0s 6ms/step -
accuracy: 0.7536 - loss: 0.6129
Epoch 76: val_accuracy improved from 0.76566 to 0.76846, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      153s 7ms/step
- accuracy: 0.7536 - loss: 0.6129 - val_accuracy: 0.7685 - val_loss: 0.5761 -
learning_rate: 6.2500e-05
Epoch 77/100
22111/22116      0s 6ms/step -
accuracy: 0.7531 - loss: 0.6138
Epoch 77: val_accuracy improved from 0.76846 to 0.77055, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      153s 7ms/step
- accuracy: 0.7531 - loss: 0.6138 - val_accuracy: 0.7705 - val_loss: 0.5728 -
learning_rate: 6.2500e-05
Epoch 78/100
22111/22116      0s 6ms/step -
accuracy: 0.7551 - loss: 0.6107
Epoch 78: val_accuracy improved from 0.77055 to 0.77142, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      153s 7ms/step
- accuracy: 0.7551 - loss: 0.6107 - val_accuracy: 0.7714 - val_loss: 0.5702 -
learning_rate: 6.2500e-05
Epoch 79/100
22111/22116      0s 6ms/step -

```

```

accuracy: 0.7553 - loss: 0.6096
Epoch 79: val_accuracy did not improve from 0.77142
22116/22116      153s 7ms/step
- accuracy: 0.7553 - loss: 0.6096 - val_accuracy: 0.7696 - val_loss: 0.5740 -
learning_rate: 6.2500e-05
Epoch 80/100
22112/22116      0s 6ms/step -
accuracy: 0.7560 - loss: 0.6084
Epoch 80: val_accuracy improved from 0.77142 to 0.77146, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116      153s 7ms/step
- accuracy: 0.7560 - loss: 0.6084 - val_accuracy: 0.7715 - val_loss: 0.5702 -
learning_rate: 6.2500e-05
Epoch 81/100
22114/22116      0s 6ms/step -
accuracy: 0.7571 - loss: 0.6062
Epoch 81: val_accuracy did not improve from 0.77146
22116/22116      153s 7ms/step
- accuracy: 0.7571 - loss: 0.6062 - val_accuracy: 0.7711 - val_loss: 0.5700 -
learning_rate: 6.2500e-05
Epoch 82/100
22115/22116      0s 6ms/step -
accuracy: 0.7564 - loss: 0.6079
Epoch 82: val_accuracy did not improve from 0.77146
22116/22116      152s 7ms/step
- accuracy: 0.7564 - loss: 0.6079 - val_accuracy: 0.7707 - val_loss: 0.5737 -
learning_rate: 6.2500e-05
Epoch 83/100
22111/22116      0s 6ms/step -
accuracy: 0.7561 - loss: 0.6077
Epoch 83: val_accuracy did not improve from 0.77146
22116/22116      153s 7ms/step
- accuracy: 0.7561 - loss: 0.6077 - val_accuracy: 0.7698 - val_loss: 0.5736 -
learning_rate: 6.2500e-05
Epoch 84/100
22115/22116      0s 6ms/step -
accuracy: 0.7570 - loss: 0.6076
Epoch 84: val_accuracy did not improve from 0.77146

Epoch 84: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
22116/22116      152s 7ms/step
- accuracy: 0.7570 - loss: 0.6076 - val_accuracy: 0.7700 - val_loss: 0.5741 -
learning_rate: 6.2500e-05
Epoch 85/100
22116/22116      0s 6ms/step -
accuracy: 0.7581 - loss: 0.6022
Epoch 85: val_accuracy improved from 0.77146 to 0.77226, saving model to
/kaggle/working/best_transformer_model.keras

```

```

22116/22116          152s 7ms/step
- accuracy: 0.7581 - loss: 0.6022 - val_accuracy: 0.7723 - val_loss: 0.5656 -
learning_rate: 3.1250e-05
Epoch 86/100
22116/22116          0s 6ms/step -
accuracy: 0.7587 - loss: 0.5991
Epoch 86: val_accuracy improved from 0.77226 to 0.77550, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          154s 7ms/step
- accuracy: 0.7587 - loss: 0.5991 - val_accuracy: 0.7755 - val_loss: 0.5597 -
learning_rate: 3.1250e-05
Epoch 87/100
22115/22116          0s 6ms/step -
accuracy: 0.7603 - loss: 0.5983
Epoch 87: val_accuracy did not improve from 0.77550
22116/22116          154s 7ms/step
- accuracy: 0.7603 - loss: 0.5983 - val_accuracy: 0.7749 - val_loss: 0.5608 -
learning_rate: 3.1250e-05
Epoch 88/100
22114/22116          0s 6ms/step -
accuracy: 0.7604 - loss: 0.5977
Epoch 88: val_accuracy did not improve from 0.77550
22116/22116          154s 7ms/step
- accuracy: 0.7604 - loss: 0.5977 - val_accuracy: 0.7749 - val_loss: 0.5606 -
learning_rate: 3.1250e-05
Epoch 89/100
22113/22116          0s 6ms/step -
accuracy: 0.7598 - loss: 0.5990
Epoch 89: val_accuracy did not improve from 0.77550

Epoch 89: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.
22116/22116          155s 7ms/step
- accuracy: 0.7598 - loss: 0.5990 - val_accuracy: 0.7741 - val_loss: 0.5607 -
learning_rate: 3.1250e-05
Epoch 90/100
22112/22116          0s 6ms/step -
accuracy: 0.7607 - loss: 0.5973
Epoch 90: val_accuracy improved from 0.77550 to 0.77660, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          156s 7ms/step
- accuracy: 0.7607 - loss: 0.5973 - val_accuracy: 0.7766 - val_loss: 0.5570 -
learning_rate: 1.5625e-05
Epoch 91/100
22113/22116          0s 6ms/step -
accuracy: 0.7629 - loss: 0.5934
Epoch 91: val_accuracy improved from 0.77660 to 0.77734, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          157s 7ms/step

```

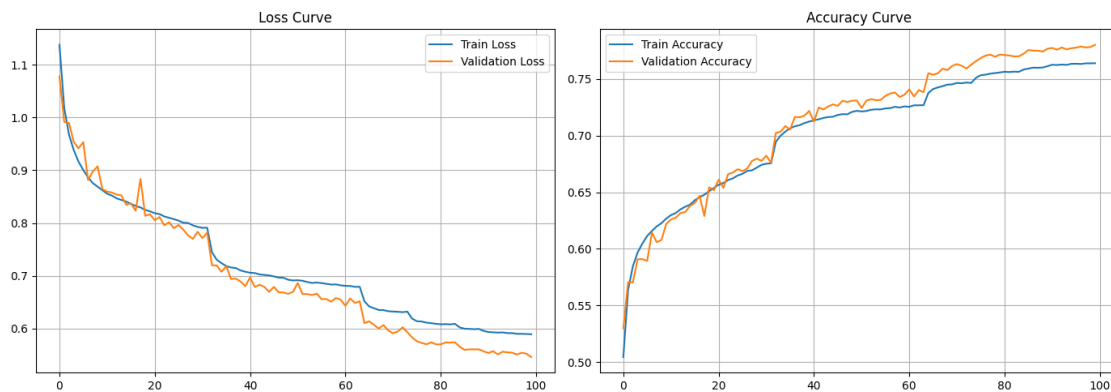
```

- accuracy: 0.7629 - loss: 0.5934 - val_accuracy: 0.7773 - val_loss: 0.5538 -
learning_rate: 1.5625e-05
Epoch 92/100
22109/22116          0s 6ms/step -
accuracy: 0.7621 - loss: 0.5930
Epoch 92: val_accuracy did not improve from 0.77734
22116/22116          155s 7ms/step
- accuracy: 0.7621 - loss: 0.5930 - val_accuracy: 0.7757 - val_loss: 0.5573 -
learning_rate: 1.5625e-05
Epoch 93/100
22112/22116          0s 6ms/step -
accuracy: 0.7629 - loss: 0.5924
Epoch 93: val_accuracy improved from 0.77734 to 0.77774, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          155s 7ms/step
- accuracy: 0.7629 - loss: 0.5924 - val_accuracy: 0.7777 - val_loss: 0.5511 -
learning_rate: 1.5625e-05
Epoch 94/100
22110/22116          0s 6ms/step -
accuracy: 0.7624 - loss: 0.5925
Epoch 94: val_accuracy did not improve from 0.77774
22116/22116          155s 7ms/step
- accuracy: 0.7624 - loss: 0.5925 - val_accuracy: 0.7761 - val_loss: 0.5563 -
learning_rate: 1.5625e-05
Epoch 95/100
22116/22116          0s 6ms/step -
accuracy: 0.7629 - loss: 0.5919
Epoch 95: val_accuracy did not improve from 0.77774
22116/22116          154s 7ms/step
- accuracy: 0.7629 - loss: 0.5919 - val_accuracy: 0.7770 - val_loss: 0.5550 -
learning_rate: 1.5625e-05
Epoch 96/100
22115/22116          0s 6ms/step -
accuracy: 0.7626 - loss: 0.5929
Epoch 96: val_accuracy did not improve from 0.77774

Epoch 96: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.
22116/22116          154s 7ms/step
- accuracy: 0.7626 - loss: 0.5929 - val_accuracy: 0.7776 - val_loss: 0.5543 -
learning_rate: 1.5625e-05
Epoch 97/100
22108/22116          0s 6ms/step -
accuracy: 0.7632 - loss: 0.5895
Epoch 97: val_accuracy improved from 0.77774 to 0.77858, saving model to
/kaggle/working/best_transformer_model.keras
22116/22116          154s 7ms/step
- accuracy: 0.7632 - loss: 0.5895 - val_accuracy: 0.7786 - val_loss: 0.5508 -
learning_rate: 7.8125e-06

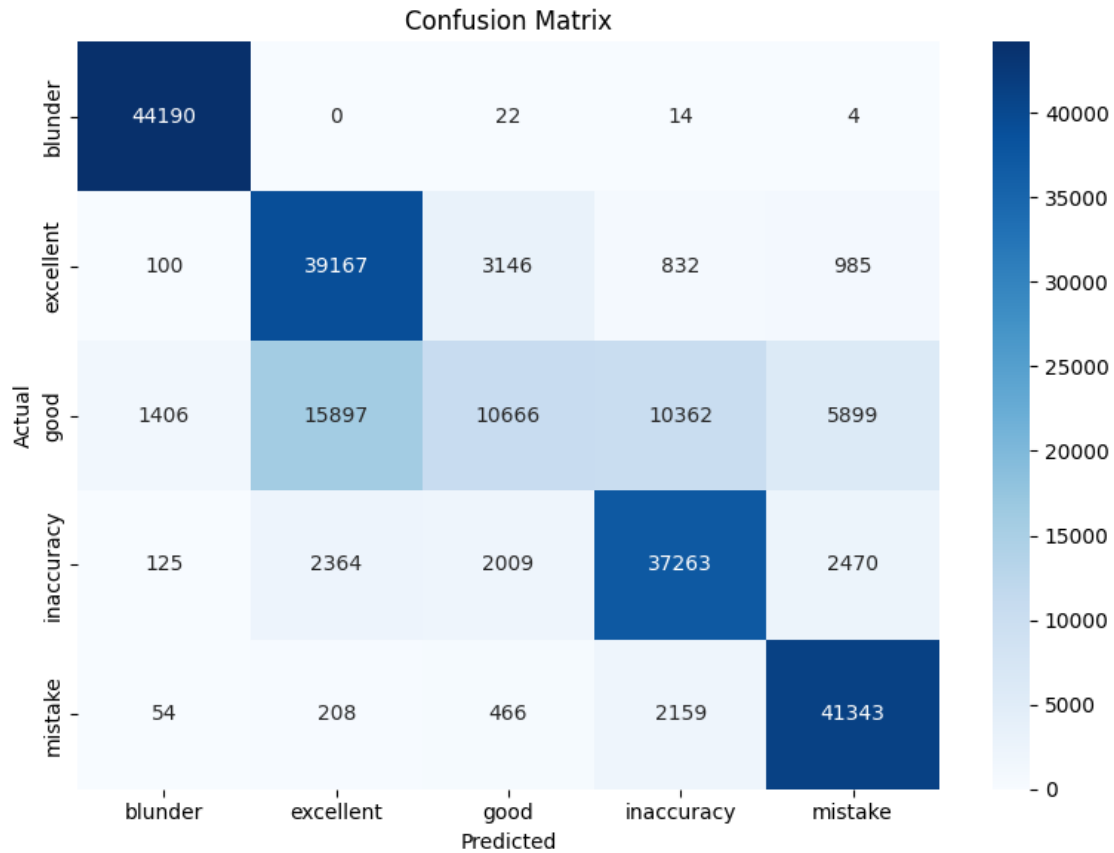
```

Epoch 98/100
 22108/22116 0s 6ms/step -
 accuracy: 0.7640 - loss: 0.5894
 Epoch 98: val_accuracy did not improve from 0.77858
 22116/22116 155s 7ms/step
 - accuracy: 0.7640 - loss: 0.5894 - val_accuracy: 0.7778 - val_loss: 0.5543 -
 learning_rate: 7.8125e-06
 Epoch 99/100
 22115/22116 0s 6ms/step -
 accuracy: 0.7648 - loss: 0.5876
 Epoch 99: val_accuracy did not improve from 0.77858
 22116/22116 156s 7ms/step
 - accuracy: 0.7648 - loss: 0.5876 - val_accuracy: 0.7781 - val_loss: 0.5524 -
 learning_rate: 7.8125e-06
 Epoch 100/100
 22108/22116 0s 6ms/step -
 accuracy: 0.7643 - loss: 0.5889
 Epoch 100: val_accuracy improved from 0.77858 to 0.78010, saving model to
 /kaggle/working/best_transformer_model.keras
 22116/22116 156s 7ms/step
 - accuracy: 0.7643 - loss: 0.5889 - val_accuracy: 0.7801 - val_loss: 0.5461 -
 learning_rate: 7.8125e-06



Evaluating model...

Test Accuracy: 78.06%
 864/864 10s 10ms/step



Classification Report:

	precision	recall	f1-score	support
blunder	0.96	1.00	0.98	44230
excellent	0.68	0.89	0.77	44230
good	0.65	0.24	0.35	44230
inaccuracy	0.74	0.84	0.79	44231
mistake	0.82	0.93	0.87	44230
accuracy			0.78	221151
macro avg	0.77	0.78	0.75	221151
weighted avg	0.77	0.78	0.75	221151

Random 100 Sample Accuracy: 76.00%

Random Samples:

	move_number	turn	en_passant	halfmove_clock	is_check	\
0	47.000000	1.000000e+00	-1.0	3.000000e+00	9.603676e-10	
1	23.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
2	121.000000	1.000000e+00	-1.0	1.600000e+01	9.603676e-10	
3	52.000000	1.532653e-08	-1.0	7.887668e-08	9.603676e-10	
4	31.000000	1.000000e+00	-1.0	3.000000e+00	9.603676e-10	
5	7.000000	1.000000e+00	-1.0	4.000000e+00	9.603676e-10	
6	53.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
7	7.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
8	33.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
9	81.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
10	23.000000	1.000000e+00	-1.0	3.000000e+00	9.603676e-10	
11	45.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
12	60.000000	1.532653e-08	-1.0	3.000000e+00	9.603676e-10	
13	97.000000	1.000000e+00	-1.0	2.000000e+00	9.603676e-10	
14	99.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
15	21.000000	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
16	8.999999	1.000000e+00	-1.0	7.887668e-08	9.603676e-10	
17	9.999999	1.532653e-08	-1.0	2.000000e+00	9.603676e-10	
18	57.000000	1.000000e+00	-1.0	2.000000e+00	9.603676e-10	
19	23.000000	1.000000e+00	-1.0	2.000000e+00	9.603676e-10	

	mobility	material_balance	prev_eval	prev_delta	elo_diff	\
0	47.000000	4.000000e+00	425.000000	209.000000	20.000000	
1	41.000000	2.000000e+00	10000.000000	9553.000000	266.999969	
2	22.000000	-7.000000e+00	-643.000000	9357.000000	-112.999992	
3	41.000000	9.999999e-01	-302.000000	-37.999985	-352.000000	
4	36.000000	9.999999e-01	34.000000	331.000000	63.000000	
5	37.000000	1.482091e-09	-53.000000	5.000000	-35.000000	
6	26.000000	-3.000000e+00	-218.000000	39.000000	-193.000000	
7	33.000000	-1.000000e+00	-48.000000	-8.000000	53.000000	
8	31.000000	-5.000000e+00	-456.000000	146.000000	-228.000000	
9	5.000000	-2.000000e+00	-475.000000	-5.999985	-143.000000	
10	39.000000	9.999999e-01	287.000000	292.000000	9.000000	
11	41.000000	-1.000000e+00	422.000000	378.000000	-160.000000	
12	37.000000	1.482091e-09	-71.000015	-37.000015	87.000000	
13	13.000001	-3.000000e+00	-590.000000	9410.000000	-36.000004	
14	6.000000	-1.800000e+01	-1117.000000	-216.000015	-230.999985	
15	32.000000	1.482091e-09	29.000002	51.000000	-196.000000	
16	45.000000	1.482091e-09	13.000002	19.000000	-59.000000	
17	33.000000	1.482091e-09	76.000000	-18.000000	-3.000000	
18	16.000000	1.482091e-09	28.000002	-5.000000	21.000000	
19	34.000000	1.482091e-09	115.999992	58.000000	-128.000000	

	avg_elo	castle_K	castle_Q	castle_k	castle_q	\
0	2316.0	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09	
1	1348.5	9.999999e-01	1.000000e+00	1.000000e+00	1.000000e+00	
2	2363.5	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09	

3	1979.0	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
4	1814.5	9.999999e-01	-5.190014e-09	-8.196377e-09	-7.831716e-09
5	1560.5	9.999999e-01	1.000000e+00	1.000000e+00	1.000000e+00
6	1486.5	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
7	1520.5	9.999999e-01	1.000000e+00	1.000000e+00	1.000000e+00
8	1225.0	-7.343750e-09	-5.190014e-09	1.000000e+00	-7.831716e-09
9	1625.5	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
10	1653.5	-7.343750e-09	-5.190014e-09	-8.196377e-09	1.000000e+00
11	1733.0	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
12	1842.5	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
13	1818.0	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
14	1809.5	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
15	1988.0	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
16	1471.5	9.999999e-01	1.000000e+00	1.000000e+00	1.000000e+00
17	1472.5	9.999999e-01	1.000000e+00	1.000000e+00	1.000000e+00
18	1806.5	-7.343750e-09	-5.190014e-09	-8.196377e-09	-7.831716e-09
19	2215.0	-7.343750e-09	-5.190014e-09	1.000000e+00	1.000000e+00

	Actual	Predicted	Correct
0	mistake	mistake	True
1	blunder	blunder	True
2	blunder	blunder	True
3	excellent	excellent	True
4	blunder	blunder	True
5	inaccuracy	inaccuracy	True
6	inaccuracy	inaccuracy	True
7	good	inaccuracy	False
8	inaccuracy	inaccuracy	True
9	good	inaccuracy	False
10	blunder	blunder	True
11	blunder	blunder	True
12	excellent	excellent	True
13	blunder	blunder	True
14	blunder	blunder	True
15	inaccuracy	inaccuracy	True
16	inaccuracy	inaccuracy	True
17	good	excellent	False
18	good	inaccuracy	False
19	inaccuracy	inaccuracy	True

=====

TRAINING COMPLETED SUCCESSFULLY

Saved files:

/kaggle/working/final_transformer_model.keras

/kaggle/working/best_transformer_model.keras

/kaggle/working/scaler.pkl

/kaggle/working/label_encoder.pkl

/kaggle/working/random_test_predictions.csv

/kaggle/working/training_history.csv

2 model summary

```
[7]: import tensorflow as tf

tf.keras.backend.clear_session()

# =====
# IMPORTS
# =====

import numpy as np

from tensorflow.keras import (
    layers,
    models,
    regularizers
)

from tensorflow.keras.utils import (
    plot_model
)

from IPython.display import (
    Image,
    display
)

# =====
# DUMMY VALUES
# (ONLY FOR BUILDING SUMMARY/PLOT)
# =====

NUM_CLASSES = 5
SCALAR_FEATURES = 15

# =====
# POSITIONAL EMBEDDING LAYER
# =====

class PositionalEmbedding(
    layers.Layer
):

    def __init__(self, sequence_length, d_model):
```

```

    super().__init__()

    self.pos_embedding = layers.Embedding(
        input_dim=sequence_length,
        output_dim=d_model
    )

    self.sequence_length = sequence_length
    self.d_model = d_model

    def call(self, inputs):

        positions = tf.range(
            start=0,
            limit=self.sequence_length,
            delta=1
        )

        embedded_positions = self.pos_embedding(
            positions
        )

        return inputs + embedded_positions

# =====
# TRANSFORMER ENCODER
# =====

def transformer_encoder(
    inputs,
    head_size,
    num_heads,
    ff_dim,
    dropout=0.1
):

    # -----
    # MULTIHEAD ATTENTION
    # -----

    x = layers.MultiHeadAttention(
        key_dim=head_size,
        num_heads=num_heads,
        dropout=dropout
    )(inputs, inputs)

```

```

# -----

x = layers.Dropout(dropout)(x)

x = layers.LayerNormalization(
    epsilon=1e-6
)(x)

# -----
# RESIDUAL CONNECTION
# -----

res = layers.Add()([
    x,
    inputs
])

# -----
# FEED FORWARD
# -----

x = layers.Dense(
    ff_dim,
    activation='swish'
)(res)

x = layers.Dropout(dropout)(x)

x = layers.Dense(
    inputs.shape[-1]
)(x)

x = layers.LayerNormalization(
    epsilon=1e-6
)(x)

# -----
# SECOND RESIDUAL
# -----

return layers.Add()([
    x,
    res
])

# =====
# BUILD MODEL

```

```

# =====
def build_model():

    # =====
    # TENSOR INPUT
    # =====

    tensor_input = layers.Input(
        shape=(12,8,8)
    )

    # =====
    # PERMUTE
    # =====

    x = layers.Permute(
        (2,3,1)
    )(tensor_input)

    # =====
    # CNN BLOCK
    # =====

    x = layers.Conv2D(
        32,
        (3,3),
        padding='same',
        activation='swish'
    )(x)

    x = layers.BatchNormalization()(x)

    x = layers.Conv2D(
        64,
        (3,3),
        padding='same',
        activation='swish'
    )(x)

    x = layers.BatchNormalization()(x)

    # =====
    # RESHAPE
    # =====

    x = layers.Reshape(

```

```

        (64,64)
    )(x)

# =====
# POSITIONAL EMBEDDING
# =====

x = PositionalEmbedding(
    sequence_length=64,
    d_model=64
)(x)

# =====
# TRANSFORMER BLOCK 1
# =====

x = transformer_encoder(
    x,
    head_size=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1
)

# =====
# TRANSFORMER BLOCK 2
# =====

x = transformer_encoder(
    x,
    head_size=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1
)

# =====
# GLOBAL AVERAGE POOLING
# =====

x = layers.GlobalAveragePooling1D()(x)

# =====
# SCALAR INPUT
# =====

scalar_input = layers.Input(

```

```

        shape=(SCALAR_FEATURES,)
    )

    y = layers.Dense(
        64,
        activation='swish'
    )(scalar_input)

    y = layers.BatchNormalization()(y)

    y = layers.Dropout(0.2)(y)

    # =====
    # CONCATENATION
    # =====

    combined = layers.concatenate(
        [x, y]
    )

    # =====
    # FINAL DENSE BLOCK
    # =====

    z = layers.Dense(
        128,
        activation='swish',
        kernel_regularizer=regularizers.l2(0.001)
    )(combined)

    z = layers.Dropout(0.3)(z)

    z = layers.Dense(
        64,
        activation='swish'
    )(z)

    z = layers.Dropout(0.3)(z)

    output = layers.Dense(
        NUM_CLASSES,
        activation='softmax'
    )(z)

    # =====
    # MODEL
    # =====

```

```

    model = models.Model(

        inputs=[
            tensor_input,
            scalar_input
        ],

        outputs=output
    )

    return model

# =====
# BUILD MODEL
# =====

model = build_model()

# =====
# MODEL SUMMARY
# =====

model.summary()

# =====
# PLOT MODEL
# =====

plot_model(

    model,

    to_file="final_transformer_model_plot.png",

    show_shapes=True,

    show_dtype=False,

    show_layer_names=True,

    rankdir="TB",

    expand_nested=False,

    dpi=120
)

```

```
# =====
# DISPLAY MODEL IMAGE
# =====

display(
    Image(
        filename="final_transformer_model_plot.png"
    )
)
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 12, 8, 8)	0	-
permute (Permute)	(None, 8, 8, 12)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 8, 8, 32)	3,488	permute[0][0]
batch_normalization (BatchNormalizatio...	(None, 8, 8, 32)	128	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 8, 8, 64)	18,496	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 8, 8, 64)	256	conv2d_1[0][0]
reshape (Reshape)	(None, 64, 64)	0	batch_normalizat...
positional_embeddi... (PositionalEmbeddi...	(None, 64, 64)	4,096	reshape[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 64, 64)	66,368	positional_embed... positional_embed...
dropout_1 (Dropout)	(None, 64, 64)	0	multi_head_atten...
layer_normalization (LayerNormalizatio...	(None, 64, 64)	128	dropout_1[0][0]
add (Add)	(None, 64, 64)	0	layer_normalizat... positional_embed...

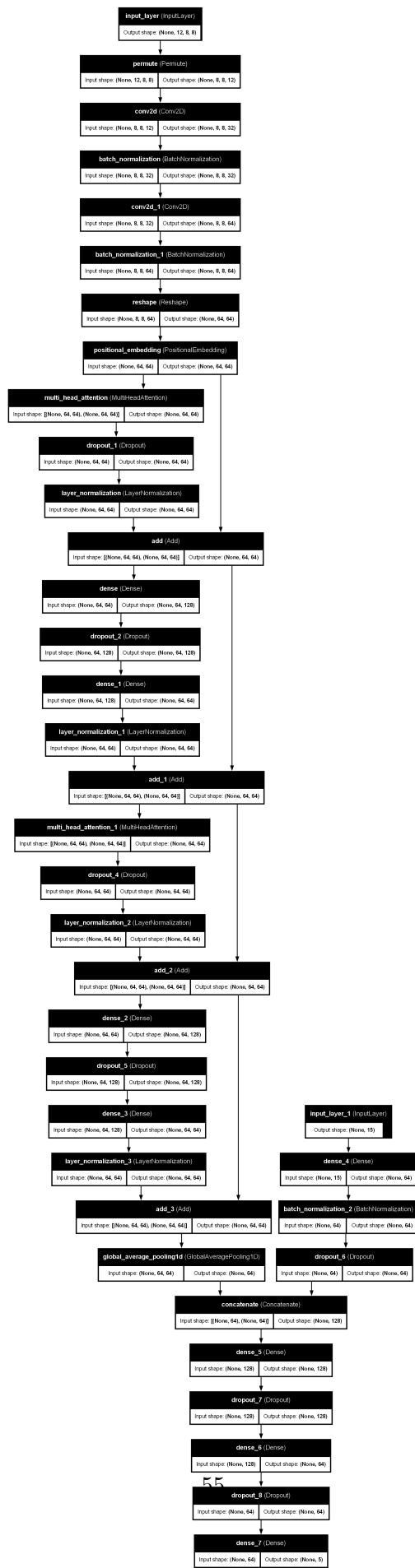
dense (Dense)	(None, 64, 128)	8,320	add[0][0]
dropout_2 (Dropout)	(None, 64, 128)	0	dense[0][0]
dense_1 (Dense)	(None, 64, 64)	8,256	dropout_2[0][0]
layer_normalization... (LayerNormalizatio...	(None, 64, 64)	128	dense_1[0][0]
add_1 (Add)	(None, 64, 64)	0	layer_normalizat... add[0][0]
multi_head_attenti... (MultiHeadAttentio...	(None, 64, 64)	66,368	add_1[0][0], add_1[0][0]
dropout_4 (Dropout)	(None, 64, 64)	0	multi_head_atten...
layer_normalization... (LayerNormalizatio...	(None, 64, 64)	128	dropout_4[0][0]
add_2 (Add)	(None, 64, 64)	0	layer_normalizat... add_1[0][0]
dense_2 (Dense)	(None, 64, 128)	8,320	add_2[0][0]
dropout_5 (Dropout)	(None, 64, 128)	0	dense_2[0][0]
dense_3 (Dense)	(None, 64, 64)	8,256	dropout_5[0][0]
input_layer_1 (InputLayer)	(None, 15)	0	-
layer_normalization... (LayerNormalizatio...	(None, 64, 64)	128	dense_3[0][0]
dense_4 (Dense)	(None, 64)	1,024	input_layer_1[0]...
add_3 (Add)	(None, 64, 64)	0	layer_normalizat... add_2[0][0]
batch_normalization... (BatchNormalizatio...	(None, 64)	256	dense_4[0][0]
global_average_poo... (GlobalAveragePool...	(None, 64)	0	add_3[0][0]
dropout_6 (Dropout)	(None, 64)	0	batch_normalizat...

concatenate (Concatenate)	(None, 128)	0	global_average_p... dropout_6[0][0]
dense_5 (Dense)	(None, 128)	16,512	concatenate[0][0]
dropout_7 (Dropout)	(None, 128)	0	dense_5[0][0]
dense_6 (Dense)	(None, 64)	8,256	dropout_7[0][0]
dropout_8 (Dropout)	(None, 64)	0	dense_6[0][0]
dense_7 (Dense)	(None, 5)	325	dropout_8[0][0]

Total params: 219,237 (856.39 KB)

Trainable params: 218,917 (855.14 KB)

Non-trainable params: 320 (1.25 KB)



```

[1]: # =====
# RESET TENSORFLOW SESSION
# IMPORTANT:
# Ensures EXACT SAME layer numbering
# =====

import tensorflow as tf

tf.keras.backend.clear_session()

# =====
# IMPORTS
# =====

import numpy as np
import pandas as pd

from tensorflow.keras import (
    layers,
    models,
    regularizers
)

from tensorflow.keras.utils import (
    plot_model
)

from IPython.display import (
    Image,
    display
)

# =====
# DUMMY SETTINGS
# ONLY FOR SUMMARY + PLOT
# =====

NUM_CLASSES = 5
SCALAR_FEATURES = 15

# =====
# POSITIONAL EMBEDDING LAYER
# =====

```

```

class PositionalEmbedding(
    layers.Layer
):

    def __init__(
        self,
        sequence_length,
        d_model
    ):

        super().__init__()

        self.pos_embedding = layers.Embedding(
            input_dim=sequence_length,
            output_dim=d_model
        )

        self.sequence_length = sequence_length
        self.d_model = d_model

    def call(self, inputs):

        positions = tf.range(
            start=0,
            limit=self.sequence_length,
            delta=1
        )

        embedded_positions = self.pos_embedding(
            positions
        )

        return inputs + embedded_positions

# =====
# TRANSFORMER ENCODER
# =====

def transformer_encoder(
    inputs,
    head_size,
    num_heads,
    ff_dim,
    dropout=0.1
):

    # =====

```

```

# MULTIHEAD ATTENTION
# =====

x = layers.MultiHeadAttention(
    key_dim=head_size,
    num_heads=num_heads,
    dropout=dropout
)(inputs, inputs)

# =====

x = layers.Dropout(
    dropout
)(x)

x = layers.LayerNormalization(
    epsilon=1e-6
)(x)

# =====
# FIRST RESIDUAL CONNECTION
# =====

res = layers.Add()([
    x,
    inputs
])

# =====
# FEED FORWARD NETWORK
# =====

x = layers.Dense(
    ff_dim,
    activation='swish'
)(res)

x = layers.Dropout(
    dropout
)(x)

x = layers.Dense(
    inputs.shape[-1]
)(x)

x = layers.LayerNormalization(
    epsilon=1e-6

```

```

)(x)

# =====
# SECOND RESIDUAL CONNECTION
# =====

return layers.Add()([
    x,
    res
])

# =====
# BUILD MODEL
# =====

def build_model():

    # =====
    # TENSOR INPUT
    # =====

    tensor_input = layers.Input(
        shape=(12,8,8)
    )

    # =====
    # PERMUTE
    # =====

    x = layers.Permute(
        (2,3,1)
    )(tensor_input)

    # =====
    # CNN BLOCK
    # =====

    x = layers.Conv2D(
        32,
        (3,3),
        padding='same',
        activation='swish'
    )(x)

    x = layers.BatchNormalization()(x)

    x = layers.Conv2D(

```

```

        64,
        (3,3),
        padding='same',
        activation='swish'
    )(x)

x = layers.BatchNormalization()(x)

# =====
# RESHAPE TO SEQUENCE
# =====

x = layers.Reshape(
    (64,64)
)(x)

# =====
# POSITIONAL EMBEDDING
# =====

x = PositionalEmbedding(
    sequence_length=64,
    d_model=64
)(x)

# =====
# TRANSFORMER BLOCK 1
# =====

x = transformer_encoder(
    x,
    head_size=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1
)

# =====
# TRANSFORMER BLOCK 2
# =====

x = transformer_encoder(
    x,
    head_size=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1

```

```

)

# =====
# GLOBAL AVERAGE POOLING
# =====

x = layers.GlobalAveragePooling1D()(x)

# =====
# SCALAR INPUT
# =====

scalar_input = layers.Input(
    shape=(SCALAR_FEATURES,)
)

y = layers.Dense(
    64,
    activation='swish'
)(scalar_input)

y = layers.BatchNormalization()(y)

y = layers.Dropout(0.2)(y)

# =====
# CONCATENATE
# =====

combined = layers.concatenate(
    [x, y]
)

# =====
# FINAL DENSE BLOCK
# =====

z = layers.Dense(
    128,
    activation='swish',
    kernel_regularizer=regularizers.l2(0.001)
)(combined)

z = layers.Dropout(0.3)(z)

z = layers.Dense(
    64,

```

```

        activation='swish'
    )(z)

    z = layers.Dropout(0.3)(z)

    output = layers.Dense(
        NUM_CLASSES,
        activation='softmax'
    )(z)

    # =====
    # MODEL
    # =====

    model = models.Model(

        inputs=[
            tensor_input,
            scalar_input
        ],

        outputs=output
    )

    return model

# =====
# BUILD MODEL
# =====

model = build_model()

# =====
# FULL MODEL SUMMARY
# =====

print("\n=====")
print("FULL TRANSFORMER MODEL SUMMARY")
print("=====\\n")

model.summary(
    line_length=140,
    expand_nested=True,
    show_trainable=True
)

# =====

```

```

# SAVE SUMMARY TABLE
# =====

summary_list = []

model.summary(
    print_fn=lambda x: summary_list.append(x)
)

summary_text = "\n".join(summary_list)

with open(
    "transformer_model_summary.txt",
    "w"
) as f:

    f.write(summary_text)

print("\nSummary table saved:")
print("transformer_model_summary.txt")

# =====
# CREATE SUMMARY DATAFRAME
# =====

rows = []

for layer in model.layers:

    rows.append({

        "Layer Name": layer.name,

        "Layer Type": layer.__class__.__name__,

        "Output Shape": str(layer.output.shape),

        "Parameters": layer.count_params()
    })

summary_df = pd.DataFrame(rows)

# =====
# DISPLAY TABLE
# =====

print("\n=====")

```

```

print("MODEL LAYERS TABLE")
print("=====\n")

print(summary_df)

# =====
# SAVE TABLE CSV
# =====

summary_df.to_csv(
    "transformer_model_layers.csv",
    index=False
)

print("\nLayer table saved:")
print("transformer_model_layers.csv")

# =====
# MODEL PLOT
# =====

plot_model(
    model,

    to_file="final_transformer_model_plot.png",

    show_shapes=True,

    show_dtype=False,

    show_layer_names=True,

    rankdir="TB",

    expand_nested=False,

    dpi=140
)

print("\nModel plot saved:")
print("final_transformer_model_plot.png")

# =====
# DISPLAY MODEL PLOT
# =====

```

```

display(
    Image(
        filename="final_transformer_model_plot.png"
    )
)

# =====
# FINAL PIPELINE
# =====

print("\n=====")
print("COMPLETE MODEL PIPELINE")
print("=====\\n")

print("""
12x8x8 Chess Tensor
    ↓
Permute Layer
    ↓
CNN Feature Extraction
(Conv2D → BN → Conv2D → BN)
    ↓
Reshape to 64x64 Sequence
    ↓
Positional Embedding
    ↓
Transformer Encoder Block 1
    ↓
Transformer Encoder Block 2
    ↓
Global Average Pooling
    ↓
64 Transformer Features

        +
        |
        | Scalar Features
        |
        | ↓
Scalar Dense Block
(Dense → BN → Dropout)
        |
        | ↓
64 Scalar Features

        |
        | ↓
Concatenation
(64 + 64 = 128 Features)
        |
        | ↓

```

```

Dense(128)
      ↓
Dropout
      ↓
Dense(64)
      ↓
Dropout
      ↓
Softmax Output
      ↓
Final Chess Move Prediction

"""

# =====
# PARAMETER COUNTS
# =====

print("=====")
print("PARAMETER STATISTICS")
print("=====\\n")

print(f"Total Parameters      : {model.count_params():,}")

trainable_params = np.sum([
    np.prod(v.shape)
    for v in model.trainable_weights
])

non_trainable_params = np.sum([
    np.prod(v.shape)
    for v in model.non_trainable_weights
])

print(f"Trainable Parameters   : {trainable_params:,}")
print(f"Non-Trainable Params   : {non_trainable_params:,}")

print("\\n=====")
print("FILES GENERATED")
print("=====\\n")

print("1. transformer_model_summary.txt")
print("2. transformer_model_layers.csv")
print("3. final_transformer_model_plot.png")

```

WARNING:tensorflow:From c:\Users\sachin kumar\.conda\envs\ML_Learning\lib\site-packages\keras\src\backend\common\global_state.py:82: The name tf.reset_default_graph is deprecated. Please use

tf.compat.v1.reset_default_graph instead.

=====

FULL TRANSFORMER MODEL SUMMARY

=====

Model: "functional"

Layer (type)	Output Shape	
↪ Param # Connected to		
input_layer (InputLayer)	(None, 12, 8, 8)	↪
↪ 0 -		
permute (Permute)	(None, 8, 8, 12)	↪
↪ 0 input_layer[0][0]		
conv2d (Conv2D)	(None, 8, 8, 32)	↪
↪ 3,488 permute[0][0]		
batch_normalization	(None, 8, 8, 32)	↪
↪ 128 conv2d[0][0]		
(BatchNormalization)		↪
↪		
conv2d_1 (Conv2D)	(None, 8, 8, 64)	↪
↪ 18,496 batch_normalization[		
batch_normalization_1	(None, 8, 8, 64)	↪
↪ 256 conv2d_1[0][0]		
(BatchNormalization)		↪
↪		
reshape (Reshape)	(None, 64, 64)	↪
↪ 0 batch_normalization_		
positional_embedding	(None, 64, 64)	↪
↪ 4,096 reshape[0][0]		
(PositionalEmbedding)		↪
↪		
multi_head_attention	(None, 64, 64)	↪
↪ 66,368 positional_embedding		

(MultiHeadAttention)		
↪ positional_embedding		
dropout_1 (Dropout)	(None, 64, 64)	
↪ 0 multi_head_attention		
layer_normalization	(None, 64, 64)	
↪ 128 dropout_1[0][0]		
(LayerNormalization)		
↪		
add (Add)	(None, 64, 64)	
↪ 0 layer_normalization[		
↪ positional_embedding		
dense (Dense)	(None, 64, 128)	
↪ 8,320 add[0][0]		
dropout_2 (Dropout)	(None, 64, 128)	
↪ 0 dense[0][0]		
dense_1 (Dense)	(None, 64, 64)	
↪ 8,256 dropout_2[0][0]		
layer_normalization_1	(None, 64, 64)	
↪ 128 dense_1[0][0]		
(LayerNormalization)		
↪		
add_1 (Add)	(None, 64, 64)	
↪ 0 layer_normalization_		
↪ add[0][0]		
multi_head_attention_1	(None, 64, 64)	
↪ 66,368 add_1[0][0], add_1[0		
(MultiHeadAttention)		
↪		
dropout_4 (Dropout)	(None, 64, 64)	
↪ 0 multi_head_attention		
layer_normalization_2	(None, 64, 64)	
↪ 128 dropout_4[0][0]		

(LayerNormalization)	(None, 64, 64)	⌋
↪		
add_2 (Add)	(None, 64, 64)	⌋
↪ 0 layer_normalization_3		
↪ add_1[0][0]		⌋
dense_2 (Dense)	(None, 64, 128)	⌋
↪ 8,320 add_2[0][0]		
dropout_5 (Dropout)	(None, 64, 128)	⌋
↪ 0 dense_2[0][0]		
dense_3 (Dense)	(None, 64, 64)	⌋
↪ 8,256 dropout_5[0][0]		
input_layer_1 (InputLayer)	(None, 15)	⌋
↪ 0 -		
layer_normalization_3	(None, 64, 64)	⌋
↪ 128 dense_3[0][0]		
(LayerNormalization)		⌋
↪		
dense_4 (Dense)	(None, 64)	⌋
↪ 1,024 input_layer_1[0][0]		
add_3 (Add)	(None, 64, 64)	⌋
↪ 0 layer_normalization_3		
↪ add_2[0][0]		⌋
batch_normalization_2	(None, 64)	⌋
↪ 256 dense_4[0][0]		
(BatchNormalization)		⌋
↪		
global_average_pooling1d	(None, 64)	⌋
↪ 0 add_3[0][0]		
(GlobalAveragePooling1D)		⌋
↪		
dropout_6 (Dropout)	(None, 64)	⌋
↪ 0 batch_normalization_2		

```

concatenate (Concatenate)                (None, 128)
↳      0  global_average_pooli
↳
↳      dropout_6[0][0]
↳
dense_5 (Dense)                          (None, 128)
↳  16,512  concatenate[0][0]
↳
dropout_7 (Dropout)                      (None, 128)
↳      0  dense_5[0][0]
↳
dense_6 (Dense)                          (None, 64)
↳      8,256  dropout_7[0][0]
↳
dropout_8 (Dropout)                      (None, 64)
↳      0  dense_6[0][0]
↳
dense_7 (Dense)                          (None, 5)
↳      325  dropout_8[0][0]

```

Total params: 219,237 (856.39 KB)

Trainable params: 218,917 (855.14 KB)

Non-trainable params: 320 (1.25 KB)

Summary table saved:
transformer_model_summary.txt

```

=====
MODEL LAYERS TABLE
=====

```

	Layer Name	Layer Type	Output Shape \
0	input_layer	InputLayer	(None, 12, 8, 8)
1	permute	Permute	(None, 8, 8, 12)
2	conv2d	Conv2D	(None, 8, 8, 32)
3	batch_normalization	BatchNormalization	(None, 8, 8, 32)
4	conv2d_1	Conv2D	(None, 8, 8, 64)
5	batch_normalization_1	BatchNormalization	(None, 8, 8, 64)
6	reshape	Reshape	(None, 64, 64)
7	positional_embedding	PositionalEmbedding	(None, 64, 64)

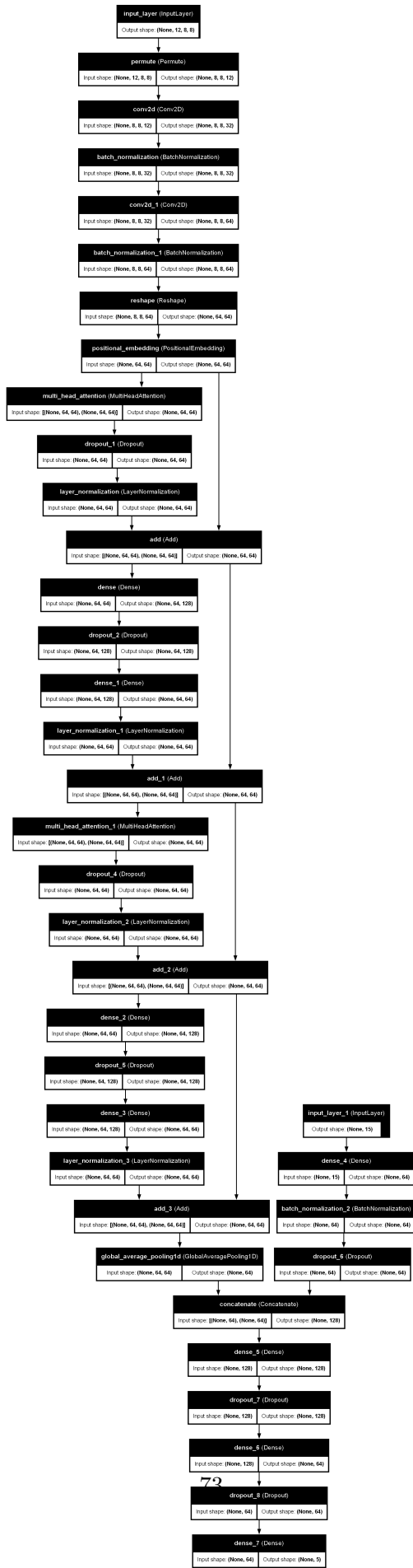
8	multi_head_attention	MultiHeadAttention	(None, 64, 64)
9	dropout_1	Dropout	(None, 64, 64)
10	layer_normalization	LayerNormalization	(None, 64, 64)
11	add	Add	(None, 64, 64)
12	dense	Dense	(None, 64, 128)
13	dropout_2	Dropout	(None, 64, 128)
14	dense_1	Dense	(None, 64, 64)
15	layer_normalization_1	LayerNormalization	(None, 64, 64)
16	add_1	Add	(None, 64, 64)
17	multi_head_attention_1	MultiHeadAttention	(None, 64, 64)
18	dropout_4	Dropout	(None, 64, 64)
19	layer_normalization_2	LayerNormalization	(None, 64, 64)
20	add_2	Add	(None, 64, 64)
21	dense_2	Dense	(None, 64, 128)
22	dropout_5	Dropout	(None, 64, 128)
23	dense_3	Dense	(None, 64, 64)
24	input_layer_1	InputLayer	(None, 15)
25	layer_normalization_3	LayerNormalization	(None, 64, 64)
26	dense_4	Dense	(None, 64)
27	add_3	Add	(None, 64, 64)
28	batch_normalization_2	BatchNormalization	(None, 64)
29	global_average_pooling1d	GlobalAveragePooling1D	(None, 64)
30	dropout_6	Dropout	(None, 64)
31	concatenate	Concatenate	(None, 128)
32	dense_5	Dense	(None, 128)
33	dropout_7	Dropout	(None, 128)
34	dense_6	Dense	(None, 64)
35	dropout_8	Dropout	(None, 64)
36	dense_7	Dense	(None, 5)

Parameters	
0	0
1	0
2	3488
3	128
4	18496
5	256
6	0
7	4096
8	66368
9	0
10	128
11	0
12	8320
13	0
14	8256
15	128
16	0

17	66368
18	0
19	128
20	0
21	8320
22	0
23	8256
24	0
25	128
26	1024
27	0
28	256
29	0
30	0
31	0
32	16512
33	0
34	8256
35	0
36	325

Layer table saved:
transformer_model_layers.csv

Model plot saved:
final_transformer_model_plot.png



=====

COMPLETE MODEL PIPELINE

=====

12x8x8 Chess Tensor
↓
Permute Layer
↓
CNN Feature Extraction
(Conv2D → BN → Conv2D → BN)
↓
Reshape to 64x64 Sequence
↓
Positional Embedding
↓
Transformer Encoder Block 1
↓
Transformer Encoder Block 2
↓
Global Average Pooling
↓
64 Transformer Features

+

Scalar Features

↓

Scalar Dense Block
(Dense → BN → Dropout)
↓
64 Scalar Features

↓

Concatenation
(64 + 64 = 128 Features)
↓
Dense(128)
↓
Dropout
↓
Dense(64)
↓
Dropout

↓
Softmax Output
↓
Final Chess Move Prediction

=====

PARAMETER STATISTICS

=====

Total Parameters : 219,237
Trainable Parameters : 218,917
Non-Trainable Params : 320

=====

FILES GENERATED

=====

1. transformer_model_summary.txt
2. transformer_model_layers.csv
3. final_transformer_model_plot.png

[]: